



WOLLO UNIVERSITY

KIOT

COLLEGE OF INFORMATICS

DEPARTMENT OF SOFTWARE ENGINEERING

Group Assignment for Requirement Engineering

Name	Id Number
ABEBE GETACHEW	WOUR/0044/16
BLEN MESFIN	WOUR/0510/16
DAGIM DEMISSEW	WOUR/0528/16
EYADERADDIS BELETE	WOUR/0749/16
FEREHAN AHMED	WOUR/0791/16
MEKLIT MULUGETA	WOUR/1286/16

Submitted to: Instructor Bihonegn A.

Submission date: 29/12/2025 G.C.

Wollo University – KIOT

Software Requirements Specification

for

Genius Guide - E-Learning

Smart Study Assistance

System

Version 1.0

Prepared by Group Three(3)

Date Created: 25th December 2025 (G.C.)

Table of Contents

Table of Contents.....	2
1. Introduction.....	3
1.1 Purpose.....	3
1.2 Document Conventions.....	4
1.3 Intended Audience Reading Suggestions.....	5
1.4 Definitions, Acronyms, & Abbreviations.....	5
1.5 Overview of the Document.....	5
2. Overall Description.....	6
2.1 Product Perspective.....	6
2.2 Product Features & Functions.....	7
2.3 User Classes and Characteristics.....	8
2.4 General Constraints.....	9
2.5 User Documentation.....	10
2.6 Assumptions and Dependencies.....	10
3. Specific Requirements.....	11
3.1 User Requirements.....	11
3.2 System Requirements.....	22
4. External Interface Requirements.....	31
4.1 User Interfaces.....	31
4.2 Hardware Interfaces.....	32
4.3 Software Interfaces.....	34
4.4 Communications Interfaces.....	37
5. Analysis Models.....	39
5.1 System Use Case Model.....	39
5.2 Sequence Diagrams.....	40
5.3 Activity Diagram.....	46
6. References.....	55
7. Appendices.....	56

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Informatics E-Learning Platform. This system is a specialized educational portal designed to serve the entire Informatics Department, encompassing four major programs: Software Engineering (SE), Information Technology (IT), Computer Science (CS), and Information Systems (IS). The platform facilitates a structured learning environment where courses are categorized by departments. It provides interactive features including video lectures, modular course materials, automated quizzes, coding practice environments (compilers), and an integrated AI assistant to enhance the student learning experience.

Scope

The system will include user authentication, course management by academic year, progress tracking, integrated coding compiler, quiz systems with rewards, and a simple AI chatbot. This SRS covers the complete web application to be developed using PHP and MySQL.

The system will provide a centralized hub for students and instructors within the Informatics department. Key modules include:

- User Management: Role-based access for Students, Instructors, and Department Admins.
- Multi-Departmental Architecture: Separate yet integrated pathways for SE, IT, CS, and IS students.
- Progress Tracking: Visual dashboards for students to monitor completed modules and grades.
- Interactive Tools: An integrated web-based compiler for coding assignments and a reward-based quiz system.
- AI Integration: A chatbot to answer academic queries and provide technical guidance.

- **Technology Stack:** The application will be developed using PHP for server-side logic and MySQL for database management, hosted via the XAMPP environment for initial deployment.

1.2 Document Conventions

- High Priority requirements are marked with (H)
- Medium Priority requirements are marked with (M)
- Low Priority requirements are marked with (L)
- Requirements are uniquely identified with IDs. REQ-F-XXX for functional requirements and REQ-NF-XXX for non-functional requirements.
- Each requirement statement carries its own independent priority.
- Technical terms are defined in Section 1.4

Priority Levels:

- **High Priority (H):** Critical features required for the MVP (Minimum Viable Product).
- **Medium Priority (M):** Important features that enhance usability but are not critical for launch.
- **Low Priority (L):** "Nice-to-have" features or future enhancements.

Identification: * REQ-F-XXX: Functional Requirements.

- REQ-NF-XXX: Non-Functional Requirements.

1.3 Intended Audience Reading Suggestions

- **Developers:** Focus on Sections 3 (Specific Requirements) and 4 (External Interfaces)
- **Project Managers:** Focus on Sections 2 (Overall Description) and 3.1 (User Requirements)
- **Testers:** Focus on Section 3 (all requirements for test case development)
- **System Administrator:** Focus on Sections 1, 2, and administrative requirements.
- **End Users (Students):** Focus on Section 3.1.1 (Functional Requirements)

Reading Suggestions

Begin with Section 1 (Introduction), then Section 2 (Overall Description), followed by Section 3 (Specific Requirements) based on your role.

1.4 Definitions, Acronyms, & Abbreviations

- SE: Software Engineering
- SRS: Software Requirements Specification
- UI: User Interface
- UX: User Experience
- IDE: Integrated Development Environment
- MCQ: Multiple Choice Question
- API: Application Programming Interface
- PHP: Hypertext Preprocessor (server-side scripting language)
- MySQL: Database management system
- XAMPP: Cross-platform web server solution stack

1.5 Overview of the Document

This document is organized into seven main sections:

1. Section 1: Introduction and Scope.
2. Section 2: Overall System Description.
3. Section 3: Specific Functional and Non-Functional Requirements.
4. Section 4: External Interface Requirements.
5. Section 5: Analysis Models (Data Flow Diagrams and Entity Relationship Diagrams).
6. Section 6: References.
7. Section 7: Appendices.

2. Overall Description

2.1 Product Perspective

The **Genius Guide Smart Study Assistant** is envisioned as a foundational **Learning Management System (LMS)** designed specifically for the **College of Informatics** within the Higher Education context. It serves as a unified digital ecosystem that consolidates the academic resources of four distinct departments: **Software Engineering (SE)**, **Computer Science (CS)**, **Information Technology (IT)**, and **Information Systems (IS)**.

Currently, the distribution of learning materials, tracking of student progress, and coding practice rely on fragmented, manual methods . such as flash drives, disparate file-sharing groups, and local compiler installations. The Genius Guide platform replaces these disjointed methods with a centralized, web-based environment.

Technically, the system operates as a **hybrid web application**. While it functions as a standalone platform for user authentication, progress tracking, and curriculum management, it acts as an intelligent aggregator for resource-heavy tasks. By integrating with **YouTube** for video streaming and External code execution service for cloud-based code execution, the platform delivers high-performance features without requiring expensive, high-end server infrastructure. This perspective allows the system to remain lightweight and deployable on free hosting solutions while still offering enterprise-level features.

2.2 Product Features & Functions

The platform is designed with a modular architecture to cater to the diverse needs of the four informatics departments. The major functional groups include:

1. Multi-Departmental Curriculum Hub

- **Departmental Segregation:** Upon registration, users select their specific field of study (SE, CS, IT, or IS). The system dynamically filters content to show only relevant courses.
- **Academic Year Organization:** Content is structured hierarchically by academic level (Year 2 through Year 5), ensuring students access material appropriate for their current standing.

2. Integrated Development Environment (IDE)

- **Cloud-Based Coding:** A built-in code editor supports various programming languages (C++, Java, Python). This feature is critical for **SE and CS** students for algorithm practice, while providing necessary support for **IT and IS** students during their introductory programming courses.
- **Real-Time Compilation:** Leveraging the JDoodle API, the system compiles code and returns output instantly within the browser, removing the need for local compiler configuration.

3. Interactive Learning & Assessment

- **Multimedia Integration:** The system embeds video lectures and renders text-based notes (PDF/HTML) for a multimodal learning experience.
- **Module-Based Quizzes:** Each learning unit concludes with an automated quiz to validate understanding.
- **Gamification Engine:** To drive engagement, the system awards "Coins" and "Badges" for quiz performance and streak maintenance.

4. Intelligent Student Support

- **AI Study Assistant:** A chatbot interface is available 24/7 to answer academic queries, providing immediate support when instructors are unavailable.
- **Visual Analytics:** A "Dashboard" provides students with progress bars for every course and a "Resume" button to pick up exactly where they left off.

5. Administrative Control Panel

- **Content Management:** Instructors can upload, edit, and delete materials specific to their department.
- **User & Progress Monitoring:** Admins can view analytics on student performance to identify those who may be falling behind.

2.3 User Classes and Characteristics

The system is designed to accommodate users with varying roles and technical capabilities across the college:

1. Students (Primary Users)

- **Profile:** Undergraduate students enrolled in the College of Informatics (SE, CS, IT, IS).
- **Characteristics:**
 - **SE & CS Students:** Typically possess higher technical proficiency; will heavily utilize the Code Editor and advanced algorithmic quizzes.
 - **IT & IS Students:** Focus more on system concepts and applications; will rely heavily on video lectures and theoretical quizzes, using the code editor for core programming requirements.
- **Goal:** To access curriculum materials, practice coding skills, and track academic progress.

2. System Administrators (Content Managers)

- **Profile:** Undergraduate students enrolled in the College of Informatics (SE, CS, IT, IS).
- **Characteristics:** Technologically proficient users responsible for system integrity, course data management, and user administration.

- **Goal:** to manage the platform, upload and organize course materials, manage user accounts, and monitor system engagement metrics.

3. Guests (Prospective Users)

- **Profile:** Freshmen, external visitors, or students deciding on their major.
- **Characteristics:** Curious users with no login credentials.
- **Goal:** To browse the course catalog and view the syllabus structure without accessing the core learning materials or taking quizzes.

2.4 General Constraints

The architectural design and deployment strategy are shaped by specific environmental, economic, and technical boundaries to ensure the project is feasible for the university context:

2.4.1. Technological Compliance & Standardization

To ensure long-term maintainability and alignment with the web programming curriculum taught within the college, the backend is strictly mandated to use **PHP**, paired with a **MySQL** database. This ensures that future cohorts of students can maintain the legacy code. Additionally, the development workflow is standardized around **XAMPP/WAMP** to ensure consistency across local testing environments.

2.4.2 Zero-Cost Operational Mandate

The project is executed under a strict zero-budget constraint. Consequently, the architecture relies entirely on **free-tier services**. The system must be optimized to run on free hosting providers (e.g., **000webhost**, **InfinityFree**) and utilize free APIs for AI and code compilation. This prohibits the use of paid server instances or premium software licenses.

2.4.3 Legacy Hardware Optimization

The system acts as a "Thin Client" to accommodate the university's existing infrastructure. School laboratory computers often utilize older hardware with limited processing power. By offloading heavy tasks (like video hosting to YouTube and compilation to the Cloud API), the software ensures smooth performance even on low-spec machines.

2.4.4. Browser & Platform Independence

To guarantee accessibility for all students, the application is purely web-based. It requires **Cross-Browser Compatibility** (specifically **Google Chrome** and **Mozilla Firefox**) and must function without requiring students to install plugins or software extensions, which is often restricted on lab computers.

2.4.5 Time-Boxed Delivery

The entire Software Development Life Cycle (SDLC)—from requirements gathering to final deployment is constrained to the current **academic semester**. This necessitates a prioritization of core features (MVP) over "nice-to-have" additions, with weekly sprint reviews to ensure progress alignment.*

2.5 User Documentation

To ensure successful adoption across the four departments, the following documentation will be delivered:

1. **Student User Manual:** A step-by-step guide on account creation, department selection, and using the IDE.
2. **Administrator Guide:** Instructions for the system administrator regarding content uploads, quiz creation, user management, and system configuration.
3. **Contextual Online Help:** Tooltips and pop-up guides integrated directly into the user interface (e.g., "How to use the compiler").
4. **Deployment & Installation Guide:** A documentation set for future IT administrators detailing how to deploy the PHP application and import the MySQL database schema.

2.6 Assumptions and Dependencies

Assumptions

1. **Network Infrastructure:** It is assumed that the university computer labs and student personal devices have a stable internet connection to facilitate API calls and video streaming.
2. **Content Provision:** The system provides the structure, but it is assumed that faculty from SE, CS, IT, and IS will provide the actual content (lecture notes and videos).
3. **Client Capability:** It is assumed that user devices support modern HTML5 standards (Canvas, Video Tags).

Dependencies

1. **YouTube Integration:** The video learning module is entirely dependent on YouTube's continued availability and embedding policies.
2. **JDoodle API Stability:** The coding practice environment depends on the uptime and free-tier limits of the JDoodle code execution API.
3. **Hosting Environment:** The system depends on the selected free hosting provider supporting the specific versions of PHP and MySQL used during development.
4. **Frontend Frameworks:** The User Interface relies on a responsive Front-End Framework library to ensure responsiveness across mobile and desktop devices.

3. Specific Requirements

3.1 User Requirements

3.1.1 Functional User Requirements

3.1.1.1 User Authentication & Management

Requirement ID	Requirement	Description	Source	Priority	Related Requirements
REQ-F-001	User Registration	New users must be able to create an account with email, password, and level selection	Student needs	H	REQ-F-002, REQ-F-003
REQ-F-002	User Login	Registered users must be able to log in with email and password	Student needs	H	REQ-F-001, REQ-F-004
REQ-F-003	Password Recovery	Users must be able to reset forgotten passwords via email	Student needs	M	REQ-F-001
REQ-F-004	Profile Management	Users must be able to view and edit their profile information	Student needs	M	REQ-NF-034

3.1.1.2 Course Management

Requirement ID	Requirement	Description	Source	Priority	Related Requirements
REQ-F-005	Course Organization by level	Courses must be organized by academic level	Department structure	H	REQ-F-006, REQ-F-007
REQ-F-006	Course Enrollment	Students must be able to enroll in courses for their year	Student needs	H	REQ-F-005, REQ-F-008
REQ-F-007	Course Display	System must display for authenticated students available courses with descriptions and requirements	Student needs	H	REQ-F-005

3.1.1.3 Learning System

Requirement ID	Requirement	Description	Source	Priority	Related Requirements
REQ-F-008	Sequential Content Access	Users must complete current module (video + quiz) before accessing next module	Learning Pedagogy	H	REQ-F-009, REQ-F-015
REQ-F-009	Video Lecture Playback	System must play video lectures with standard controls (play, pause, volume)	Student needs	H	REQ-F-008
REQ-F-010	Text Materials	System must display available courses with descriptions and requirements	Student needs	H	REQ-F-008
REQ-F-011	Progress Tracking	System must track and display user progress (percentage completed) for each course	Student needs	H	REQ-F-008, REQ-F-016
REQ-F-012	Resume Functionality	Users must be able to resume from where they left off in a course	Student needs	H	REQ-F-011
REQ-F-013	Module Quizzes	Each module must end with a quiz that users must pass to continue	Learning Assessment	H	REQ-F-008, REQ-F-014

REQ-F-014	Quiz Feedback	System must provide immediate feedback: celebration for correct answers, explanations for incorrect answers	Learning Pedagogy	H	REQ-F-013
REQ-F-015	Competitive Quizzes	Separate section for timed competitive quizzes with leaderboards	Engagement Strategy	M	REQ-NF-032, REQ-NF-033
REQ-F-016	Quiz History	System must store and display user's quiz history and scores	Student needs	M	REQ-F-013, REQ-F-015

3.1.1.4 Coding Practice System

Requirement ID	Requirement	Description	Source	Priority	Related Requirements
REQ-F-017	Integrated Code Editor	System must provide a code editor with syntax highlighting for programming practice	Informatics focus	H	REQ-F-018, REQ-F-019
REQ-F-018	Code Execution	Users must be able to run code and see output within the platform	Informatics focus	H	REQ-F-017

REQ-F-019	Practice Exercises	System must provide coding exercises linked to programming courses	Learning application	H	REQ-F-017
REQ-F-020	Temporary Workspace	Code written in practice area is session-based (not permanently saved)	Technical constraint	M	REQ-F-017

3.1.1.5 AI Assistant & Gamification

Requirement ID	Requirement	Description	Source	Priority	Related Requirements
REQ-F-021	Chat Interface	System must provide a chat interface for students to ask questions	Informatics focus	M	REQ-F-022
REQ-F-022	Pre-Written Responses	System must respond to common questions with pre-programmed answers using keyword matching	Student support	M	REQ-F-021

REQ-F-023	Virtual Currency (coins)	System must award coins for quiz performance and course completion	Student support	M	REQ-F-024, REQ-F-025
REQ-F-024	Achievement Badges	System must award badges for milestones and achievements	Engagement strategy	M	REQ-F-023, REQ-NF-034
REQ-F-025	Premium Course Access	Users must be able to use coins to access premium courses	Engagement strategy	M	REQ-F-023

3.1.1.6 Administrative Features (System Administrator)

Requirement ID	Requirement	Description	Source	Priority	Related Requirements
REQ-F-026	Course Content Management	System Administrator must be able to add, edit, and remove course materials	System Admin needs	M	REQ-F-005, REQ-F-027

REQ-F-027	Quiz Management	System Administrator must be able to create and manage quiz questions	System Admin needs	M	REQ-F-013, REQ-F-026
REQ-F-028	User Management	System Administrator must be able to view and manage user accounts	System Admin needs	M	REQ-F-001
REQ-F-029	Progress Monitoring	System Administrator must be able to view student progress reports	System Admin needs	M	REQ-F-011

3.1.2 Non-Functional Requirements

3.1.2.1 Performance Requirements

Requirement ID	Requirement	Description	Source	Priority	Related Requirements
REQ-NF-030	Acceptable Performance	System should work without noticeable lag under normal school lab conditions	User experience	H	REQ-NF-031
REQ-NF-031	Smooth Video Playback	Videos should play without excessive buffering on school internet	User experience	H	REQ-F-009

3.1.2.2 Usability Requirements

Requirement ID	Requirement	Description	Source	Priority	Related Requirements
REQ-NF-032	Intuitive Navigation	System should have clear menus and easy-to-find features	User experience	H	None
REQ-NF-033	Consistent UI/UX	All pages should have consistent design, layout, and interaction patterns	User experience	H	None

REQ-NF-034	Mobile Responsive	System should work on mobile phones and tablets with responsive designs	Accessibility	H	None
REQ-NF-035	Browser Compatibility	System should work on Chrome and Firefox browsers	Technical constraint	H	None

3.1.2.3 Reliability Constraints

Requirement ID	Requirement	Description	Source	Priority	Related Requirements
REQ-NF-036	Data Persistence	User progress and achievements should not be lost	System integrity	H	REQ-F-011, REQ-F-023
REQ-NF-037	Atomicity for Critical Operations	Critical operations (e.g., quiz submission, profile update) must be atomic and ensure data integrity	System Integrity / Data Consistency	H	REQ-NF-047
REQ-NF-038	Error Handling	System should display user-friendly error messages when failures occur	User experience	M	None
REQ-NF-039	Form Validation	All forms should validate inputs before submission	Data integrity	M	REQ-F-001, REQ-F-002

3.1.2.4 Security Requirements

Requirement ID	Requirement	Description	Source	Priority	Related Requirements
REQ-NF-040	Password Encryption	User passwords must be encrypted in the database	Security	M	REQ-F-001, REQ-F-002
REQ-NF-041	Session Management	System must maintain secure user sessions	Security	M	REQ-F-002
REQ-NF-042	Input Sanitization	All user inputs must be sanitized to prevent basic attacks	Security	H	REQ-NF-038
REQ-NF-043	Role Based Access Control	Different user roles (student, admin) should have appropriate access levels	Security	H	REQ-F-026, REQ-F-028
REQ-NF-044	School Lab Compatibility	System should work on typical school laboratory computers	Deployment	H	REQ-NF-035
REQ-NF-045	Code Documentation	Source code should be well-commented and organized	Development	M	None
REQ-NF-046	Easy Content Updates	System should allow easy addition of new courses and materials	Long-term use	M	REQ-F-026

3.1.2.5 Availability

Requirement ID	Requirement	Description	Source	Priority	Related Requirements
REQ-NF-04 7	System Availability during Core Hours	The system shall have an availability of $\geq 99.5\%$ during core operating hours (7 AM - 11 PM local time)	Service Level Agreement	H	None

3.2 System Requirements

This section will give a clear explanation of every functional requirement identified in Section 3.1.1. Each of these requirements has described in it the expected behavior of the system, functionalities supported to enable user input, and system output upon encountering errors. Each requirement has been stated in a testable manner.

3.2.1 User Authentication and Management

❖ REQ-F-001 User Registration

The system shall enable a new user to register an account using a valid email address, password, academic level, and selection of a department. Validations for the email address and password shall be done prior to account registration.

- **Error Handling / System Response:**

In case any of the required fields are absent or incorrect, it must display a corresponding error message for account creation.

❖ REQ-F-002 User Login

It should be possible for registered users to login with their email and password. After that, access will be given for the personalized system functionality based on the user's role.

- **Error Handling / System Response:**

In case of authentication failure, a generic alert message will be displayed. There may be temporary blocking on repeated attempts.

- ❖ **REQ-F-003 Password Recovery**

The system should enable a user to request a password reset through their registered email address. The reset link should be time-limited and secure.

- **Error Handling / System Response:**

If the email is not registered or the token is invalid/has expired, the system should give a passive result without disclosing registration information.

- ❖ **REQ-F-004 Profile Management**

The system must enable authorized users to view and edit elements of their personal profile, such as profile name and academic level.

- **Error Handling / System Response**

Any invalid or incomplete data updates must be rejected with validation feedback.

3.2.2 Course Management

❖ REQ-F-005 Course Organization by Academic Level

The system shall be able to organize and display courses based on the academic year and department of the user.

- **Error Handling / System Response:**

If the intended course is of levels other than those of the logged-in student, then system access shall be denied with an authorization request message.

❖ REQ-F-006 Enrollment for Courses

"The system will ensure that students can only register for classes offered for their level."

- **Error Handling / System Response:**

Instances of duplicate or unauthorized enrollment will be denied with the appropriate message.

❖ REQ-F-007 Course display

The system should be able to display course description, objectives, and requirements.

- **Error Handling / System Response:**

If course details are not available, it will display a default message with missing information.

3.2.3 Learning Content

❖ REQ-F-008 Sequential Content Access

The system will enforce sequential access on the learning modules, in that the user must finish the current module before accessing the next one.

- **Handling of Errors/Response of System:**

“In case a user tries to bypass a module, the user will be prevented from accessing the system, but a progress message will be displayed.”

- ❖ **REQ-F-009 Video Lecture Playback**

The system is required to have video lectures with standard play controls.

- **Error Handling / System Response:**

In case of failure during video playing, the system will display an error message and allow a retry.

- ❖ **REQ-F-010 Text Learning Materials**

The system should provide the learning content in a readable form of text.

- **Error Handling / System Response:**

Formats of unsupported contents must generate a proper error message.

- ❖ **REQ-F-011 Progress Tracking**

It should be able to monitor and show the progress for each course for the user as a percentage.

- **Error Handling / System Response:**

"If updates cannot be made, then the existent information will be saved, with the user alerted to this fact."

- ❖ **REQ-F-012 Resume Learning**

The system should enable a user to continue learning from where they left off.

- **Error Handling / System Response:**

In case there was no previous progress, it shall begin with the first module.

3.2.4 Customer Evaluation System

❖ REQ-F-013 Module Quizzes

The system should have a quiz module at the end of every module and should require a passing grade to allow progression.

- **Error Handling / System Response:**

Incomplete forms shall be defaulted and users asked to answer all questions.

❖ REQ-F-014 Quiz Feedback

"The system shall offer immediate feedback and explanations after the submission of quizzes."

- **Error Handling / System Response:**

If the feedback data is not available, the system will alert the user.

❖ REQ-F-015 Competitive Quizzes

It must be able to offer time-based competitive quizzing with leader board results.

- **Error Handling / System Response:**

The scores will be resolved by predetermined tie-breaking rules.

❖ REQ-F-016 Quiz History

The system must be capable of storing and showing users their performance and scores for each attempt at the quiz.

- **Error Handling / System Response :**

Handling missing and/or corrupted records will take place through a notification system.

3.2.5 Coding Practice

❖ REQ-F-017 Code Editor Integration

It shall have a browser-based code editor that supports syntax highlighting.

- **Error Handling / System Response:**

Unsupported programming languages shall be disabled with a warning message.

❖ REQ-F-018 Code Execution

The system will run the user-submitted code using the cloud compiler and will display the output.

- **Error Handling / System Response:**

Execution errors or timeout shall be reported to the user.

❖ REQ-F-019 Practice Exercises

The system should enable coding practice related to relevant study courses.

- **Error Handling / System Response:**

Exercises that are not available must be indicated.

❖ REQ-F-020 Temporary Workspace

The system will temporarily save code during each session.

- **Error Handling / System Response:**

Users will be notified whenever the session times out and code is erased.

3.2.6 AI Assistant and Gamification

- ❖ **REQ-F-021 AI Chat Interface**

The system must allow for an academic-related chat service.

- **Error Handling / System Response:**

If the AI service is down, it is mandatory to notify the user.

- ❖ **REQ-F-022 Predefined Responses**

The response to common questions shall be based on keyword matching.

- **Error Handling and System Responses:**

For queries not covered, a default answer for help will be sent.

- ❖ **REQ-F-023 Virtual Currency**

"The system should be able to award coins based on the results of the quizzes and the completion of the courses."

- **Error Handling / System Response:**

Coin updates should be executed atomically.

- ❖ **REQ-F-024 Achievement Badges**

The system should reward badges on completion of milestones.

- **Error Handling/System Response:**

A duplicate badge assignment should be avoided.

- ❖ **REQ-F-025 Premium Content Access**

The system will also provide an opportunity for unlocking premium content through coins.

- **Error Handling / System Response:**

If it is inadequate, denial should be accompanied by a warning.

3.2.7 Administrative Functions

- ❖ **REQ-F-026 Course Content Management**

The system should enable the administrators to include, modify, and delete the course materials.

- **Error Handling / System Response:**

Unauthorized actions should be prevented and recorded.

- ❖ **REQ-F-027 Quiz Management**

The system is expected to provide the functionality for the creation and management of quizzes by the administrators.

- **Error Handling / System Response:**

Incorrect quiz settings will be disallowed.

- ❖ **REQ-F-028 User Management**

"The system must enable administrators to manage user accounts."

- **Error Handling / System Response:**

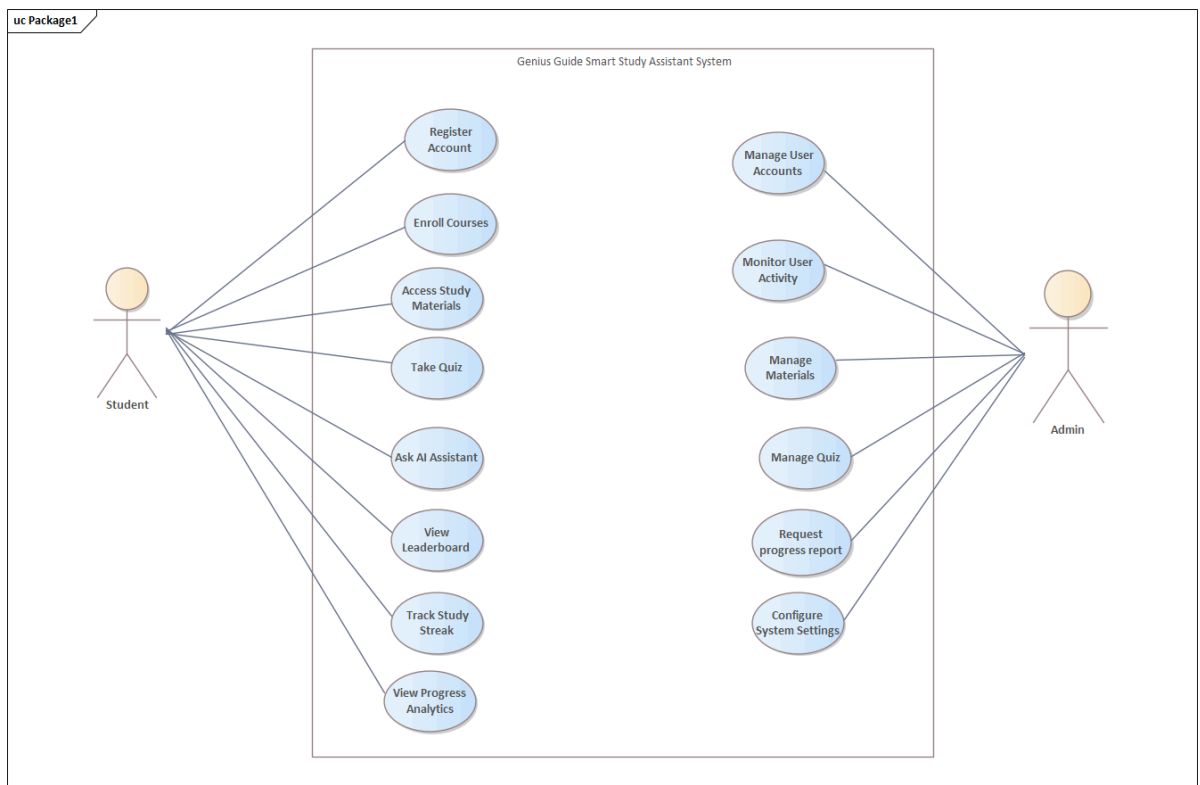
Every action will be recorded for audit trail purposes.

- ❖ **REQ-F-029 Monitoring Progress**

The system must enable viewing of progress reports of students by the administrators.

- **Error Handling / System Response:**

Omissions in data must be indicated in reports.



Essential Use Case Diagram

4. External Interface Requirements

4.1 User Interfaces

The system shall implement a web-based interface with the following characteristics:

❖ Layout and Structure:

- Primary navigation menu with items: Home, Courses, My Learning, Quizzes, Practice, Profile
- Breadcrumb navigation showing user's current location within course hierarchy
- Persistent AI assistant chat toggle button on all pages

❖ Navigation Components:

- Main Navigation Bar: Fixed position at top with user profile indicator and notifications
- Sidebar Navigation: For course modules with expandable/collapsible sections
- Quick Action Buttons: Floating action buttons for common tasks (Take Quiz, Ask AI, Practice)
- Search Functionality: Global search bar in header for course content and resources

❖ Visual Design Elements:

- Professional color scheme using institutional colors with appropriate contrast
- Clear typography hierarchy with readable font sizes
- Consistent button styles and form controls across all pages
- Visual feedback for all interactive elements (hover states, active states)

❖ AI Assistant Interface:

- Collapsible chat panel accessible from screen corner
- Clear distinction between user messages and assistant responses
- Typing indicator when assistant is "processing" responses
- History of recent conversations within current session

❖ **Reward Display Components:**

- Visual badge display on user profile with earned achievements
- Progress bars showing course completion percentages
- Celebration animations triggered by achievements and correct answers
- Coin balance display in user profile header

4.2 Hardware Interfaces

The Smart Study Assistance Platform is designed as a web-based application that does not depend on any specific hardware, meaning it can run on any consumer device using any standard browser and any cloud-based infrastructure. Since no proprietary hardware is required to use this application, users have access to many different consumer devices (e.g., desktops, laptops, tablets, smartphones) and can leverage any compatible browser when accessing the cloud servers running Smart Study Assistance. All communications between these devices and the cloud-based servers are secured using the highest level of security with HTTPS and WebSockets. The servers are built upon a scalable environment; therefore, they provide virtualized CPU and GPU resources for AI processing, as well as a distributed storage ecosystem for web-scale deployments. All specialized processing occurs in the cloud, while Smart Study Assistance delivers the same functionality to each consumer regardless of the devices they use.

4.2.1 Primary Client Device Interfaces

The software interfaces with the user's hardware via a standards-compliant web browser, which acts as an intermediary.

Supported Device Types:

- Personal Computers & Laptops, Mobile Devices (Smartphones and tablets), Input Devices (Standard keyboard, mouse, touchscreen).

Nature of Interactions:

- Data Input: Keyboard/Mouse/Touchscreen: For all standard navigation, text input in forms, quiz taking, scheduling, and chat functionality. Data is captured as browser events and processed by the web application.
- Data Output: Display/GPU Renders the graphical user interface
- Control Interactions: The software sends commands to the browser to utilize hardware APIs. User permissions are required for sensitive hardware (notifications).

Communication Protocols:

- HTTPS/HTTP2: Primary protocol for all client-server communication, ensuring encrypted data transfer for security.

4.2.2 Server-Side & External Hardware Interfaces

The application's backend interfaces with hardware in data centers (cloud infrastructure) and third-party services.

1. Cloud Server Infrastructure:

Supported Hardware Types:

- Virtualized Compute Instances (CPU/RAM), Block Storage Volumes (SSD/HDD), Load Balancers, and Content Delivery Network (CDN) edge servers.

Nature of Interactions:

- Compute: The application software operates through a virtual environment/container, where the CPU is being used for executing business logic (Quiz Grading and Artificial Intelligence processing) and the RAM will be used for caching data during a user's session.

- **Storage:** When a user creates a profile, takes a quiz, and accesses resource information the created persistent data is written to a dedicated storage area via Block Storage (Databases such as PostgreSQL, MySQL) and Blob Storage (PDF, Video, etc.).
- **Networking:** Incoming HTTPS requests are distributed across multiple servers using Load Balancing technology. Content Delivery Networks (CDN) cache static content files (Images, JavaScript, CSS) and maximize global reach by providing content from CDN to the end user location for improved latency response time.

Communication Protocols:

- **TCP/IP:** Foundation for all internal cloud network communication.

2. AI/ML Processing Hardware (via Cloud Services):

Supported Hardware Types:

- This is abstracted via cloud APIs. Underlying hardware may include GPUs (NVIDIA Tesla series, etc.) and TPUs specifically provisioned for machine learning workloads by the cloud provider (e.g., Google Cloud AI, AWS SageMaker, Azure AI Services).

Nature of Interactions:

- The AI Study Assistant and AI-Based Study Recommendations modules send structured data (user queries, quiz performance history) to these cloud AI APIs.

Communication Protocols:

- **HTTPS with RESTful/gRPC Endpoints:** Secure API calls to external AI service providers (e.g., Open router API, or custom models deployed on cloud ML platforms).

4.3 Software Interfaces

"The Smart Study Assistance Website will leverage and connect with multiple third-party software components, APIs and services to fulfil its full potential. The connectivity between the Smart Study Assistance Website and these external components is central to enabling the website's user authentication, its ability to provide artificial intelligence functionality (AI features), its ability to deliver real-time features and its ability to store user-generated content.

4.3.1 External Software Components and APIs

1. Authentication and Identity Provider Services

- ❖ Components: Google OAuth 2.0 API & an email service provider (e.g., SendGrid, AWS SES or similar email service provider) used to send transactional emails.
- ❖ Purpose: To Authenticate Users and manage User Profiles (including social login and password-reset capabilities).
- ❖ Data Transmission:
 - Incoming: User Credentials/OAuth Tokens, New User Profile data.
 - Outgoing: Authentication requests, email sent for verification/password reset.
- ❖ Method of Transmission: Via an HTTPS REST API call; the Smart Study Assistance Website will act as an OAuth 2.0 client, exchanging a user's authorization code to receive an access token from Google to obtain the user's profile data.
- ❖ Data Shared: User Identifier, Email, Name, Profile Picture URL.

2. AI/ML APIs for Services

- ❖ Components: A Large Language Model (LLM) such as Open Router AI API, Anthropic Claude, or a self-hosted LLM running in the Cloud/A.I. Platform.
- ❖ Purpose: The Purpose of the AI Study Assistant is to provide explanations and summaries of material, as well as provide recommendations based on AI.
- ❖ Data Transmission:
 - Incoming: User Text Queries, Context of User's Learning History, User's Areas of Weakness

- Outgoing: Structured prompts sent to the LLM API
- ❖ Method of Transmission: the data will be transmitted via HTTPS to either a REST or gRPC endpoint.
- ❖ Data Shared: The anonymized Text Query will be made available to other users if they have a similar proficiency score (based on their proficiency user level) and are tagged with the Subject Tags associated with the Type of Data

3. Database Management System

- ❖ Components: Primary Database: PostgreSQL (v14+) for relational data (users, quizzes, schedules). Cache & Session Store: Redis (v7+)..
- ❖ Purpose: Persistent data storage for all core functionalities.
- ❖ Data Transmission:
 - Incoming: All structured application data (CRUD operations).
 - Outgoing: Query results, user state, file streams.
- ❖ Communication: SQL over TCP/IP for PostgreSQL, Redis protocol for cache, HTTPS for S3.
- ❖ Data Shared: All application data models. Implementation Constraint: Database interactions must use connection pooling and parameterized queries to ensure performance and prevent SQL injection.

4.3.2 Data Sharing Across Internal Software Components

The application will be based upon a modular, service-oriented architecture (microservices for the back end, etc.) and enable modules to interact with each other and share information through multiple methods, including:

- ❖ Central Database: All core entities Users, Quiz, Questions, and Resources will be stored in one location that anyone can access through the application's Data Access Layer (DAL) which provides one place for all data.
- ❖ Internal APIs: Communication between modules will occur via RESTful/gRPC-endpoints of a well-defined nature, allowing for clear lines of communication between designed modules. For example, the Progress Tracker

module calls the Quiz Module's internal API for obtaining information about the user's last attempt at a Quiz, whilst the AI Study Assistant uses the Recommendation Service API to access a database for information about a user's performance.

4.4 Communications Interfaces

The Smart Study Assistance will need to have a number of different types of Communication Interfaces in order to provide the interactivity, Real-Time interaction, and AI-Driven functionality that will be required by the Client and End-Users. As well as providing for the Enabling of Secure, Reliable, and Efficient Data Exchanges between the Client and Backend Services as well as with any External Services that will be needed to work with the Application.

4.4.1 Primary Communication Protocols & Standards

1. Hypertext Transfer Protocol Secure

The main Communication Protocol used is going to be HTTPS (HTTP/1.1 and HTTP/2) for all Synchronous Client-Server Communication (i.e., loading pages, sending forms, calling APIs, etc.). The request sent by the client to the Server will be formatted with one of the Standard HTTP Methods (GET, POST, etc.) as well as the associated data, which will be sent as URL-Encoded Parameters for Non-File uploads, Multipart/Form-Data if uploading Files, and JSON when dealing with API Data.

2. Simple Mail Transfer Protocol (SMTP) / Email API (HTTPS)

SMTP or email APIs based on https are used for email communications for the purposes of confirmation, recovery, and notification. The message is composed as a MIME-compliant email and includes content in both plain-text and HTML formats. Templates are managed by the PHP backend. Connections to external email providers are secured using TLS encryption for SMTP connections and HTTPS for direct API

connections. This enables safe and reliable delivery of transactional emails and scheduled reminder e-mails.

4.4.2 Data Transfer Rates & Performance Requirements

Performance is controlled based on key speed goals for each type of interaction. The web application (including dashboard and quizzes) must deliver Largest Contentful Paint within 2.5 seconds under fast 3G conditions due to the benefit of PHP OPcache and Redis. Real-time interactions such as chat must deliver messages sub-500ms using optimized WebSocket connections. Responses from AI Study Assistant to customer inquiries must be delivered incrementally, and streamed to the user via HTTP/2 or WebSocket streaming for continued interaction when the response time exceeds three seconds.

4.4.3 Communication Security Specifications

Mandatory encryption secures all traffic between systems outside of an organization using an end-to-end model, and Communication for Services between Systems Internal is also secured using Mutual TLS & VPN or privately secured networks between service systems. Therefore, to access the Backoffice API, all authenticated users must possess either valid session/cookies for use with Backoffice API calls. Services (system external) are integrated through the use of secure API keys stored in a password manager. For Data Integrity, the following three methods are in place; Data Validation (via PHP and PHP input validation), SQL Prepared Statements, and Malware Scanning of File Uploads. Control over Cross-Origin Request (CORS) is strictly limited through the usage of precise CORS header values to limit API access to only approved Domains; therefore, controlling Data Access's confidentiality, authenticating Users/Services for Data Protection, Securing Payloads, and Preventing Unauthorized Cross-Domain Access are achieved using these multiple layers of security.

5. Analysis Models

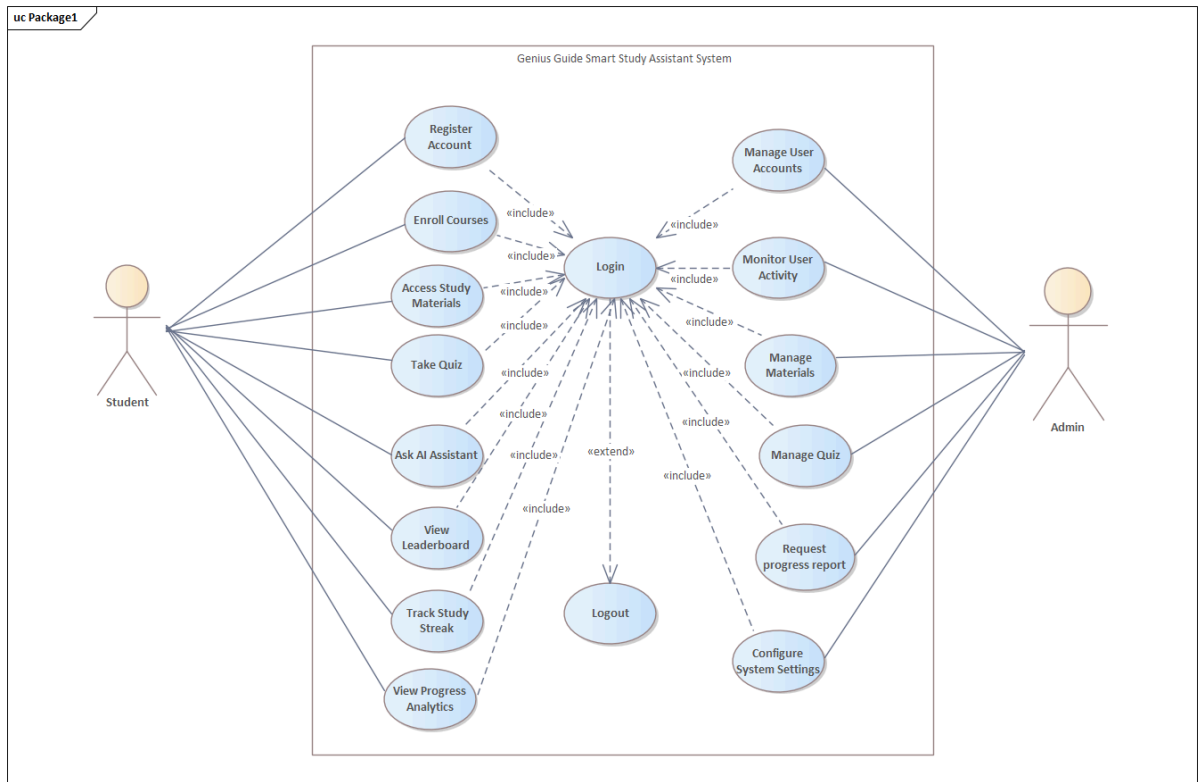
5.1 System Use Case Model

Introduction:

The Use Case Model represents all functional requirements from the user's perspective, showing interactions between actors (Students, Administrators) and the system. It defines WHAT the system does without specifying HOW.

Narrative Description:

This Use Case Model primarily defines all software functions that will be met through 13 core functional use cases. This means that any function required by the software as defined in the SRS will also be provided through one or more of these core functional use cases. Core functions will provide students with the following core learning functions: account registration, quiz taking, use of AI study assistant, visualization of academic progress, and broad access to educational material. The administrative function of the system is that all processes are managed through administrative oversight of user accounts, administrative creation and moderation of study materials, configuration management of the software, and administrative system management functions.



5.2 Sequence Diagrams

Introduction:

Sequence Diagrams model the time-ordered sequence of interactions between system components and actors to accomplish specific tasks. They are crucial for detailing the step-by-step flow of key operations, highlighting the collaboration between objects across different architectural layers (User Interface, Application Logic, Data).

Narrative Description:

This section details the operational flow for four fundamental system processes that implement key requirements from Section 3. The "Enroll in Course" diagram shows how student enrollment requests are validated against academic level and department constraints (REQ-F-005, REQ-F-006). The "Take Quiz" diagram traces assessment completion, automated grading, and reward distribution (REQ-F-013, REQ-F-023). The "Ask AI Assistance" diagram demonstrates the intelligent query routing that first checks local knowledge before engaging external AI services (REQ-F-021, REQ-F-022). Finally, the "Create Course (Admin)" diagram illustrates administrative

content management with validation and access control (REQ-F-026, REQ-NF-043). Together these diagrams provide implementable specifications for the platform's core user interactions.

Diagram 5.2.1: Sequence Diagram for "Student Enroll in Course"

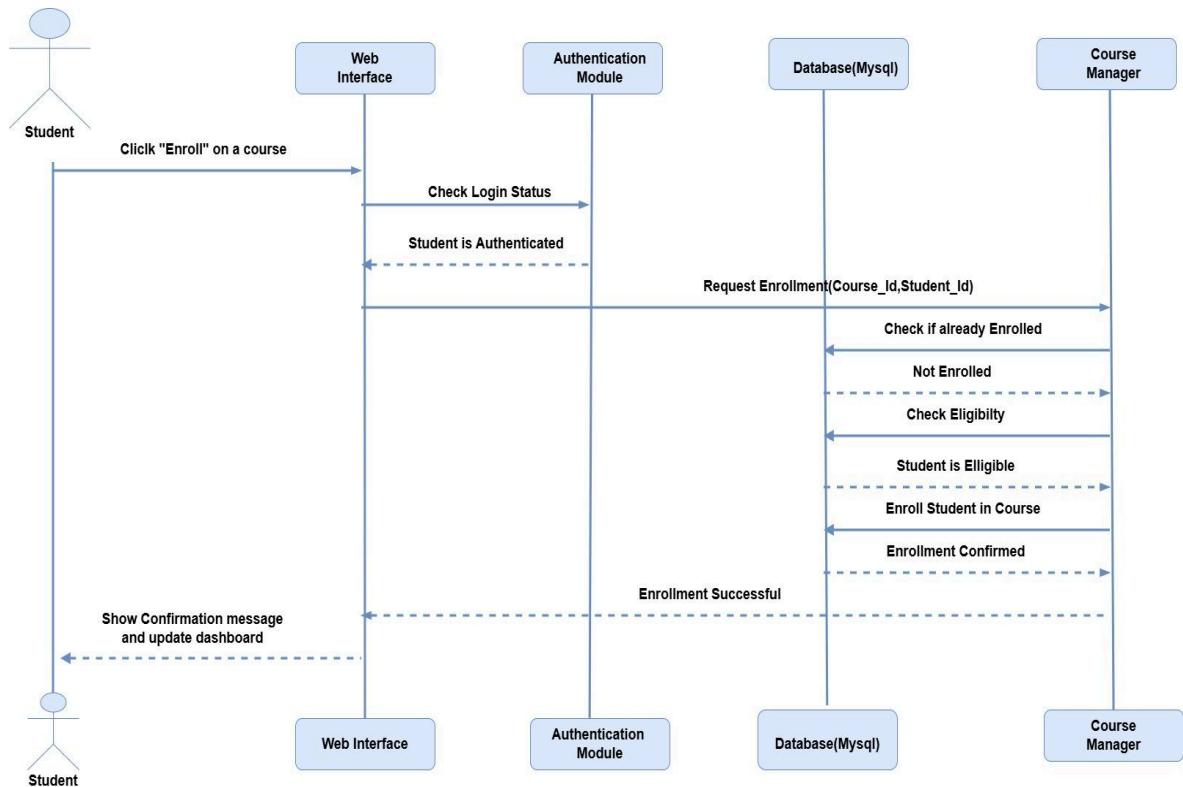


Diagram Description: This sequence diagram illustrates the "Enroll in Course" workflow, starting when a student selects a course, followed by system verification of authentication and eligibility based on academic level and department, before the Course Manager processes the enrollment in the database and confirms successful registration with updated dashboard feedback to the student.

Diagram 5.2.2: Sequence Diagram for "Student Takes a Quiz"

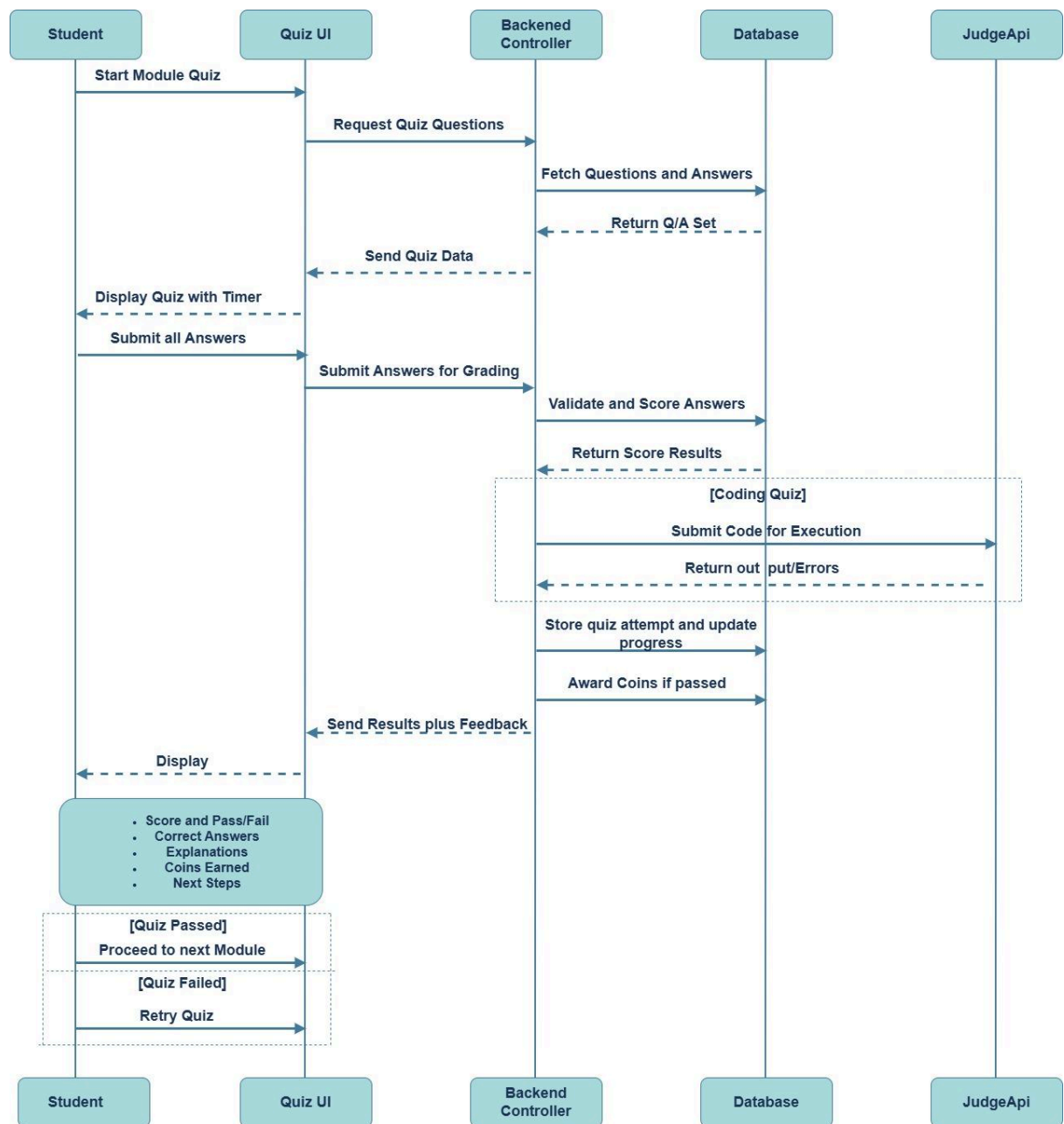


Diagram Description: This diagram details the complete lifecycle of a quiz attempt, from loading questions to submitting answers, automated grading, score calculation, and the subsequent awarding of coins. It enforces the sequential learning flow and integrates the gamification reward system.

Diagram 5.2.3: Sequence Diagram for "Student Asks AI Assistant"

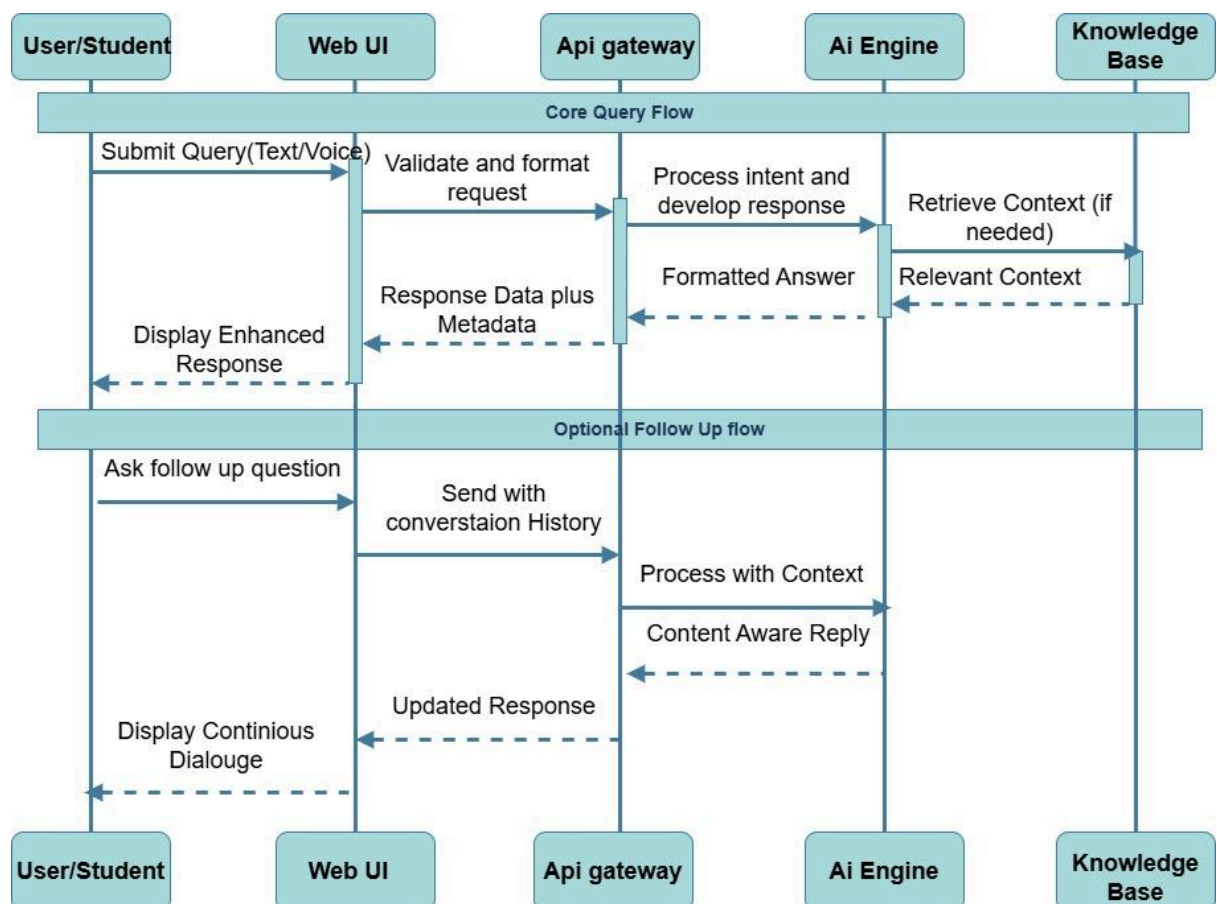


Diagram Description: This diagram models the intelligent query-handling mechanism. It shows the system's dual-path logic, first checking a local knowledge base for efficient,

predefined responses before routing complex queries to an external AI/LLM service, ensuring both responsiveness and advanced capability.

Diagram 5.2.4: Sequence Diagram for "Admin Creates a Course"

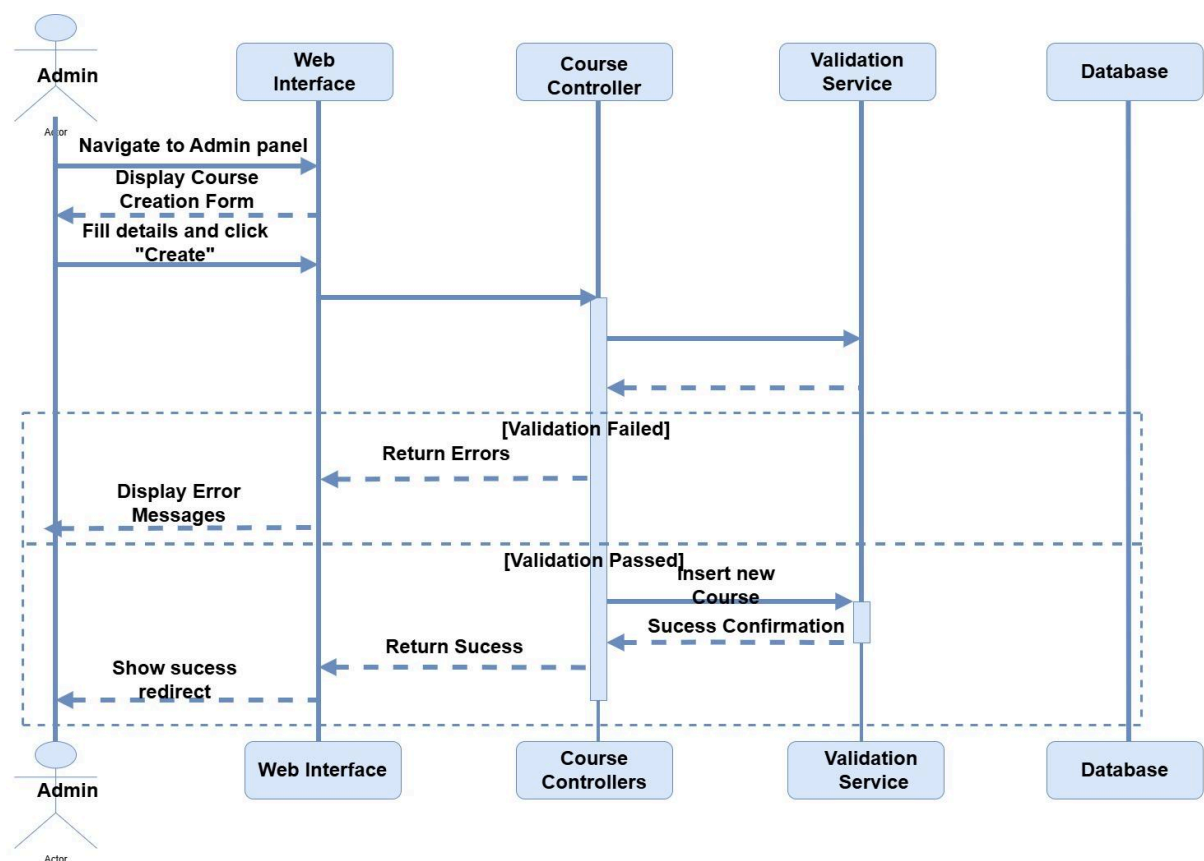


Diagram Description: This diagram outlines the administrative workflow for adding a new course. It emphasizes role-based access control, comprehensive input validation, and the final persistence of the new course entity to the database, showcasing a key content management function.

Diagram 5.2.5: Sequence diagram for “Admin Request for student progress report”

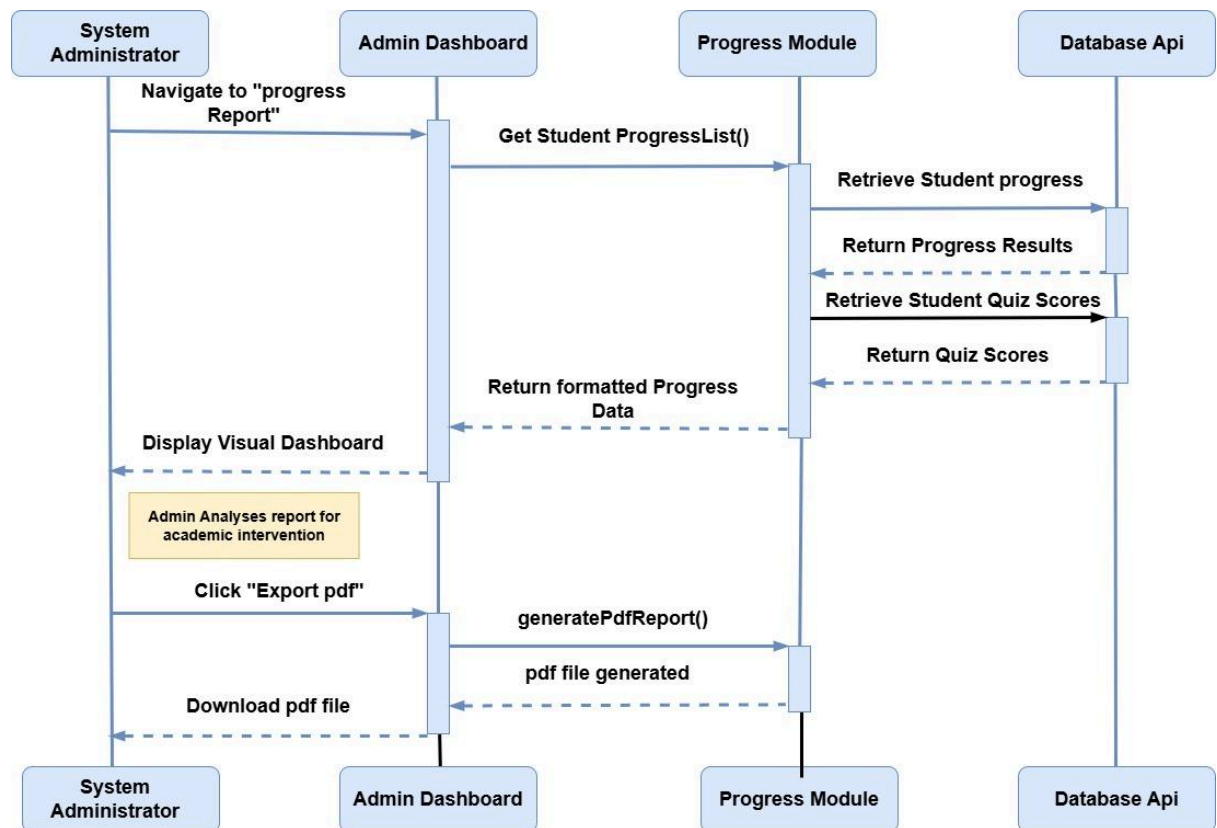


Diagram Description: This sequence diagram illustrates the chronological interaction between the System Administrator, the Admin Dashboard interface, the Progress Monitoring module, and the Database when generating a student progress report. The sequence begins with the administrator navigating to the reports section, triggering a request for student data. The Progress Module queries the database for quiz scores and module completion records, processes this information, and returns a formatted visual dashboard displaying progress percentages, performance metrics, and risk indicators. The administrator can then optionally request a PDF export, which the system generates and provides for download, completing the reporting workflow.

5.3 Activity Diagram

Introduction:

Activity Diagrams show the workflow and control flow of key processes in the system, including decision points, parallel activities, and system responses.

Diagram 5.3.1: Activity Diagram for "Student Enroll Course"

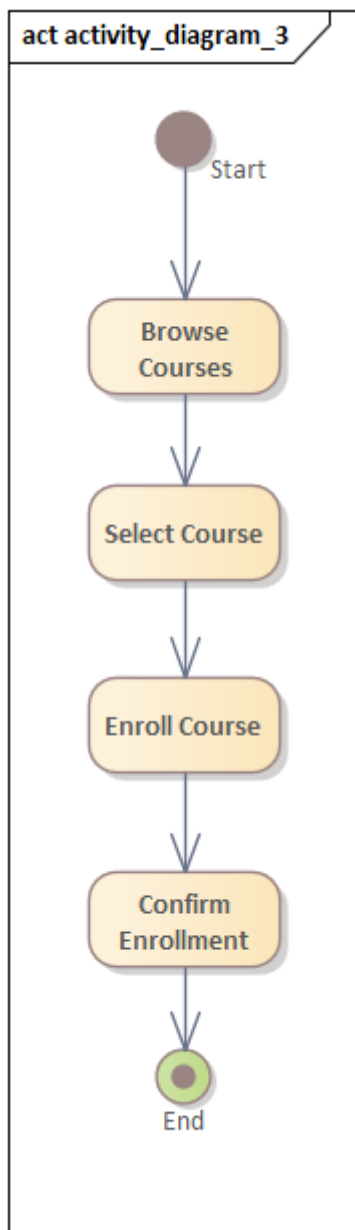


Diagram Description: This activity diagram illustrates the straightforward process for a student to enroll in a course. Starting with browsing available courses, the student selects one, proceeds to enroll, and finalizes the process with a confirmation step before the activity concludes.

Diagram 5.3.2: Activity Diagram for "Student Take Quiz"

act activity_diagram_1

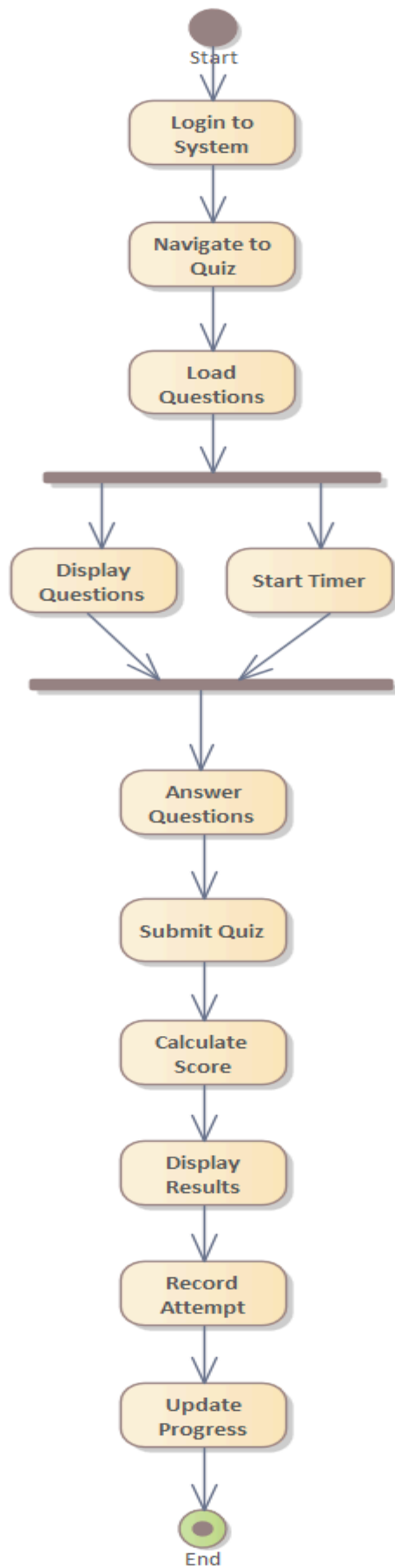


Diagram Description: This activity diagram outlines the sequential process for taking a quiz within a digital system. It begins with the user logging in and navigating to the quiz module, after which the questions are loaded and displayed alongside an active timer. The core activity involves answering the presented questions and submitting the quiz for evaluation. The process concludes with the system calculating the score, displaying the results, recording the attempt, and updating the user's progress before ending the session.

Diagram 5.3.3: Activity Diagram for "Student Ask AI Assistant"

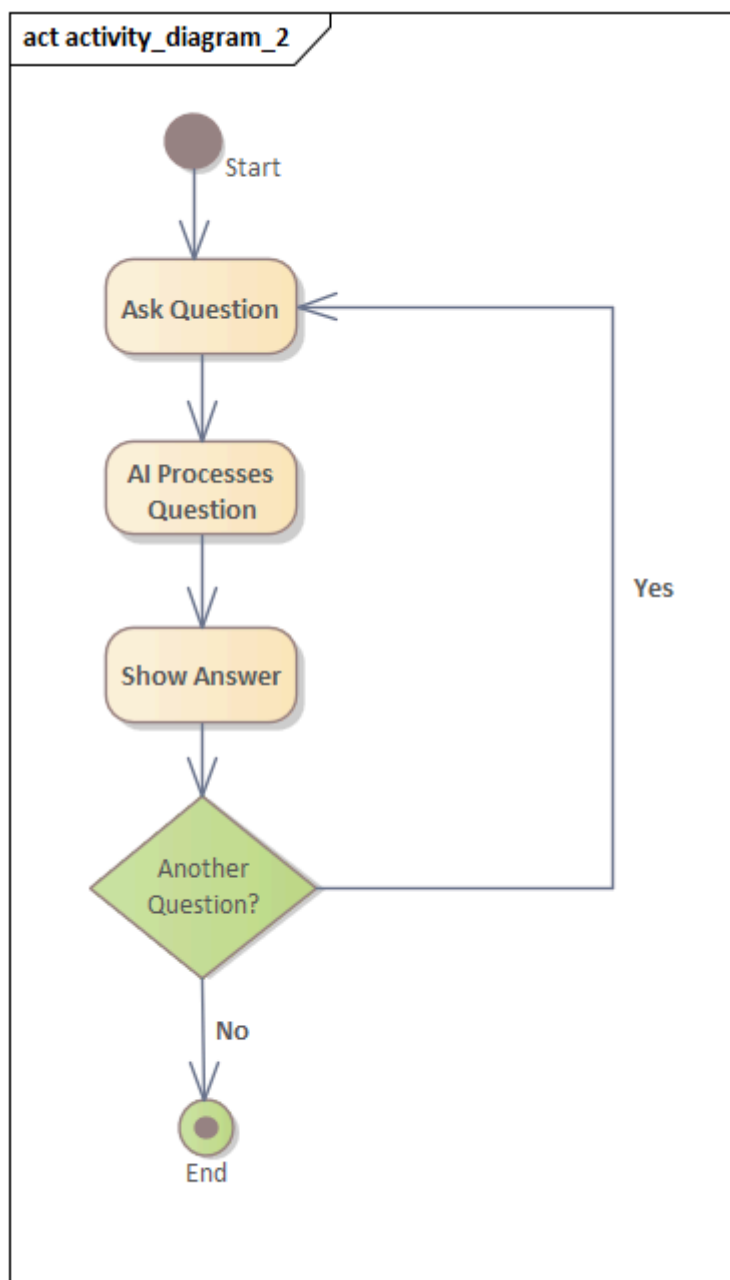


Diagram Description: This activity diagram outlines a student's interaction with an AI assistant for academic support. The process begins with the student asking questions to the AI. It then flows to the AI generating and providing a response, after which the student can review the answer and formally conclude the interaction.

Diagram 5.3.4: Activity Diagram for "Admin Creates a Course"

act activity_diagram_4

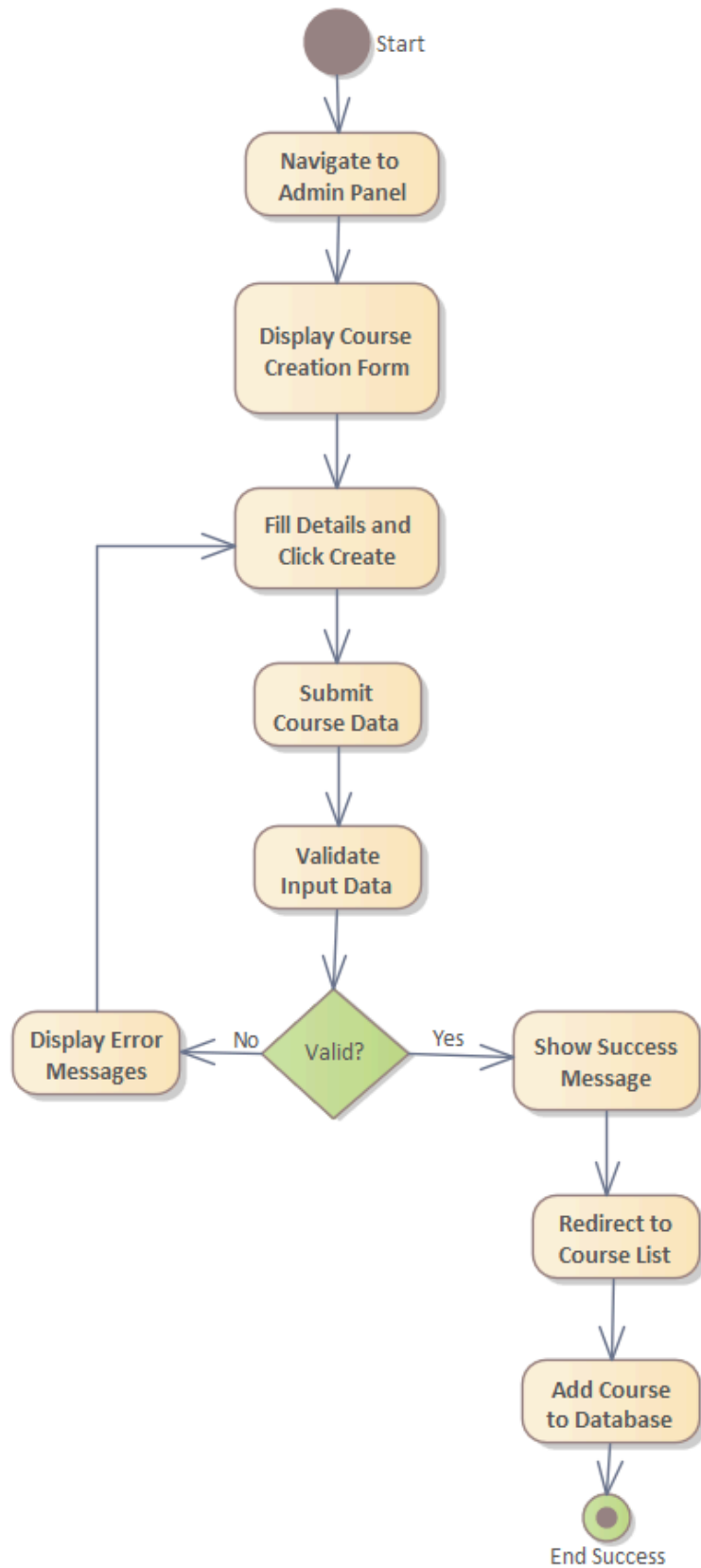


Diagram Description: This activity diagram illustrates the total procedure to create a course for an administrator within a particular system that contains a critical loop for validation and error recovery options. The initial step in the workflow is to log into the control panel and launch the course creation form to input all relevant information. After the submission is completed, the submitted information is processed through a series of validation checks where if there are validation failures, the admin receives an error message identifying the specific fields that failed validation and allows them to go back to the course creation form to correct the errors and resubmit the form. If the validation succeeds, the admin receives a success prompt and is redirected back to the course list, and a new course is permanently created and saved in the course database as a new course. This diagram accurately describes two paths for the subnet administrator and the necessary fault recovery process as envisioned by the user.

Diagram 5.3.5: Activity Diagram for "Admin request for student progress report"

act activity_diagram_5

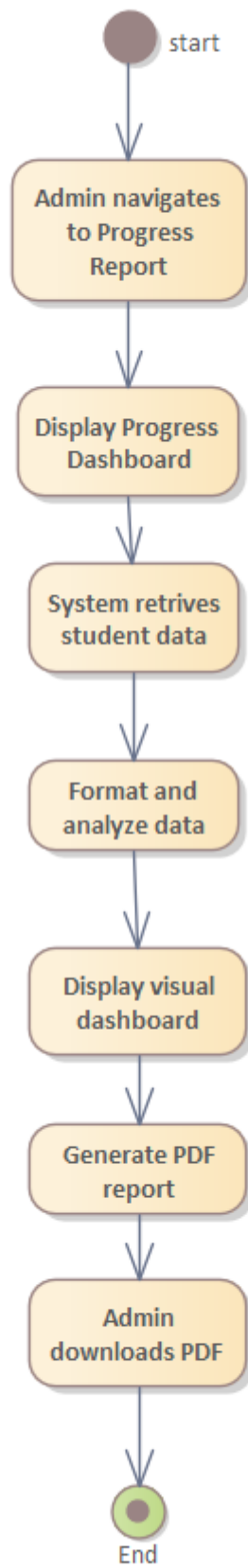


Diagram Description: The activity diagram above demonstrates how an administrator can generate a student progress report. When an administrator goes to the Progress Report section, the system displays the Progress Dashboard. After the administrator selects the Student for which they wish to generate a report, the system retrieves and organizes the necessary student information for review. Once all relevant data has been compiled and organized, it is shown as a visual dashboard and the administrator can create a PDF report by clicking the button "Generate PDF". Once the PDF has been generated, the final step in this activity is for the administrator to have the ability to download the completed PDF report.

6. References

This section lists authoritative sources, standards, and foundational documents that informed the development of this Software Requirements Specification.

6.1 Standards & Guidelines

- ISO/IEC/IEEE 29148:2018 – Systems and software engineering — Life cycle processes — Requirements engineering. International Organization for Standardization, 2018.

6.2 Technical Documentation

- PHP Documentation – The PHP Group, 2025. Available: <https://www.php.net/docs.php>
- MySQL 8.0 Reference Manual – Oracle Corporation, 2025. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- JDoodle API Documentation – JDoodle, 2025. Available: <https://jDoodle.com/docs>
- Bootstrap 5 Documentation – 2025. Available: <https://getbootstrap.com/docs/5.0/>

6.3 Institutional References

- College of Informatics Curriculum Documents and academic program outlines for SE, CS, IT, and IS departments.

6.4 Online Resources & Tools

- XAMPP Documentation – Apache Friends, 2025. Available: <https://www.apachefriends.org/docs/>

7. Appendices

7.1 Questionnaires Used to Gather Requirements

The questionnaires and surveys that were used for collecting requirements related to the Genius Guide's Smart Study Assistant platform can be found in this appendix, which contains copies of each of the individual surveys/questionnaires, which were distributed by the project team to key stakeholders for their input into the creation of the specification of requirements as contained in this document.

Questionnaire 1: Student Needs Assessment Survey

Section A: Demographic Information

- Your Department:
 - ☐ Software Engineering (SE)
 - ☐ Computer Science (CS)
 - ☐ Information Technology (IT)
 - ☐ Information Systems (IS)
- Your Academic Year:
 - ☐ Year 2
 - ☐ Year 3
 - ☐ Year 4
 - ☐ Year 5

Section B: Current Learning Experience

- How do you currently access course materials? (Select all that apply)
 - ☐ Flash drives from instructors
 - ☐ WhatsApp/Telegram groups
 - ☐ Email attachments
 - ☐ Google Drive links
 - ☐ Printed handouts

- What is your biggest challenge with current learning methods?
 - ☐ Materials are disorganized
 - ☐ No centralized platform
 - ☐ Difficult to track progress
 - ☐ No practice environment for coding

Questionnaire 2: System Administrator Requirements Interview Guide

Interview Questions:

- How should student accounts be created and managed?"
- How should course materials be organized (by department, year, etc.)?"
- What student progress data should be tracked and reported?"
- How should quiz results be stored and displayed?"
- What administrative functions are essential for managing the platform?"
- How should errors and system issues be handled?"

7.2 Glossary of Academic Terms

Terms with Definitions

- **Academic Year:** A period of time (typically two semesters) during which the curriculum is delivered.
- **Module:** A self-contained unit of learning within a course, typically comprising a video lecture, reading materials, and a quiz.
- **Departmental Segregation:** The system's ability to filter and display content specific to SE, CS, IT, or IS programs.
- **Progress Tracking:** The continuous monitoring and visualization of a student's completion status across courses.
- **Gamification Engine:** The subsystem responsible for awarding virtual coins and badges based on user achievements.

7.3 Technology Stack Justification

Technology with Justification

- **PHP:** Selected for alignment with the university's web programming curriculum, ensuring maintainability by future student cohorts.
- **MySQL:** Chosen for relational data management, compatibility with PHP, and zero licensing cost.
- **Bootstrap:** Adopted to ensure responsive design across devices without extensive custom CSS development.
- **JDoodle API:** Utilized to offload code compilation, eliminating the need for local compiler installations on lab machines.
- **YouTube:** Embed Leveraged for video hosting to reduce server storage and bandwidth requirements.

7.4 Revision History

Version	Date	Author	Changes
1.0	25 Dec 2025	Group 3	Initial release of SRS document.
0.9	20 Dec 2025	Group 3	Incorporated reviewer feedback, updated use case diagrams.
0.8	17 Dec 2025	Group 3	Added non-functional requirements and security specifications.
0.7	12 Dec 2025	Group 3	First draft of functional requirements and scope definition.